

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 222 — Past Questions & Materials
by Renance

Course: CSC 222
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 3
Levels: 200

This study pack was compiled and restructured by Renance for CSC 222 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — CSC 222 System Analysis and design 2023/2024

Level 200 • Second Semester • PastQuestion

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN,
DELTA STATE, NIGERIA.

COLLEGE OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

Second Semester Examination, 2023/2024 Session

Course Code: CSC 222

Course Title: Systems Analysis and Design Time Allowed: 2 hrs.

Instructions

Answer question one and any other three

* You are reminded of the need for good English and clear presentation in your answers.

QUESTION ONE.(COMPULSORY): {25 marks}

(a) What is system and Who is a System Analyst? (3 marks]

(©) Explain any two (2) interpersonal skills of a System Analyst [4 marks]

(c) Discuss the two (2) task performed by a System Analyst (2 marks]

(d) Briefly explain any two (2) roles of a System Analyst (2 marks]

(e) With a well labeled diagram discuss the traditional Systems Development Life Cycle (SDLC)

(10 marks]

(f) Using any four (4) points, compare Structured Systems Analysis and Design with Object Oriented Systems Analysis and Design {4 marks]

QUESTION TWO [15 marks]

(a) Explain with the aid of a diagram incremental and iterative model in detail?

(£0 marks]

(b) What is Rapid Application Development (RAD)? [2 marks]

(c) State any four (4) advantages of RAD methodology [2 marks]

(d) State two (2) disadvantages of RAD methodology (1 mark]

QUESTION THREE [15 marks] (

(a) Discuss the task regions (framework activities) of the spiral model {4 marks}

(6) State one (1) advantages and one (1) disadvantages of the spiral model (2 marks]

(c) Explain role of business analyst in detail? [6 marks] i

(a) Explain Data Analysis in detail? [3 marks]

QUESTION FOUR | {15 marks] -

(a) Discuss the three (3) common factors for feasibility study in detail [6 marks]

(b) What is the potential results of exconting a project with poor quality versus a project

executed with high quality? [4 marks]

(c) How is project execution and control different from all other work? (3 marks)

(d) List the five (5) process descriptions [2 marks]

QUESTION FIVE ~ (15 marks)

(a) What is Data Flow Diagram (DFD) [3 marks]

(b) Discuss with the aid of diagrams a Data Flow Diagram (DFD) for an ATM (Automated Teller Machine) system for banking customers. ; _ {42 marks}

QUESTION SIX , [15 marks]

(a) What is system investigation/data gathering? Write short note on the following systems investigation techniques/data gathering methods:

{4 marks]

i. Interviews

ii. Questionnaires

iii. — Written Documents Analysis

(b) Diagrammatically discuss a Work Breakdown Structure (WBS) using an aircraft system as a typical example ' [8 marks]

~(c) State any three (3) types of UML diagrams , (3 mark}

i a ---

Paper 2 of 3 — CSC 222 PQ 2024

Level 200 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN,
DELTA STATE, NIGERIA.
COLLEGE OF SCIENCE
DEPARTMENT OF COMPUTER SCIENCE
 Second Semester Examination, 2023/2024 Session

Course Code: CSC 222
 Course Title: Systems Analysis and Design
 Time Allowed: 2 hrs.

Instructions

- Answer question one and any other three
- You are reminded of the need for good English and clear presentation in your answers.

QUESTION ONE (COMPULSORY):

(a) What is system and Who is a System Analyst? [25 marks]

(b) Explain any two (2) interpersonal skills of a System Analyst [3 marks]

(c) Discuss the two (2) task performed by a System Analyst [4 marks]

(d) Briefly explain any two (2) roles of a System Analyst [2 marks]

(e) With a well labeled diagram discuss the traditional Systems Development Life Cycle (SDLC) [2 marks]

(f) Using any four (4) points, compare Structured Systems Analysis and Design with Object Oriented Systems Analysis and Design [10 marks]

(g) [4 marks]

QUESTION TWO [15 marks]

(a) Explain with the aid of a diagram incremental and iterative model in detail? [10 marks]

(b) What is Rapid Application Development (RAD)? [2 marks]

(c) State any four (4) advantages of RAD methodology [2 marks]

(d) State two (2) disadvantages of RAD methodology [1 mark]

QUESTION THREE [15 marks]

(a) Discuss the task regions (framework activities) of the spiral model [4 marks]

(b) State one (1) advantages and one (1) disadvantages of the spiral model [2 marks]

(c) Explain role of business analyst in detail? [6 marks]

(d) Explain Data Analysis in detail? [3 marks]

Paper 3 of 3 — CSC 222 SOFTWARE REUSUABILITY

Level 200 • Second Semester • Handout

CSC 222: SOFTWARE REUSUABILITY.

SOFTWARE SYSTEM PROCESSES

Software system processes encompass the structured set of activities, methods, and practices used to develop and maintain software. These processes, often organized into a Software Development Life Cycle (SDLC), ensure efficiency and quality throughout the software's creation and lifespan.

Key stages include:

Requirement gatherings

design

development

testing

deployment

and maintenance.

Key Aspects of Software System Processes:

□ SDLC (Software Development Life Cycle):

A framework that outlines the stages involved in developing and maintaining software.

□ Process Models:

Different approaches to organizing and executing the SDLC, such as Waterfall, Agile, and Iterative models.

□ Process Activities:

Specific tasks within the SDLC, including requirements analysis, design, coding, testing, and deployment.

□ Software Maintenance:

Modifying and updating software after deployment to correct errors or enhance functionality.

□ System Software:

Software that manages computer hardware and provides a platform for application software.

Common Software Process Activities:

1. Requirements Analysis: Defining what the software should do.

2. System Design: Creating a blueprint for the software's structure and components.
3. Development/Coding: Writing the actual code for the software.
4. Testing: Ensuring the software functions correctly and meets requirements.
5. Deployment: Making the software available for users.
6. Maintenance: Ongoing support and updates for the software.

Examples of Software Process Models:

□ Waterfall Model:

A linear, sequential approach where each stage is completed before the next begins.

□ Agile Model:

An iterative and incremental approach that emphasizes flexibility and collaboration.

□ Iterative Model:

A model that involves building the software in stages, with each cycle adding functionality.

□ Spiral Model:

A risk-driven approach that combines elements of the waterfall and prototyping models.

SOFTWARE DESIGN METHODS AND DESIGN EVOLUTION.

Software design methods have evolved from rigid, linear approaches like the Waterfall model to more flexible, iterative methods like Agile and Extreme Programming. This evolution reflects the increasing complexity of software projects and the need to adapt to changing requirements and user needs. Understanding the different design methods and how they evolve is crucial for building effective and maintainable software.

Traditional (Waterfall) vs. Agile/Iterative Approaches:

□ Waterfall:

This traditional model follows a sequential, linear path with distinct phases (requirements, design, implementation, testing, deployment, maintenance). Each phase must be completed before the next can begin. While it offers a structured approach, it struggles

with changing requirements and unexpected issues.

□ Agile and Evolutionary Design:

Agile methodologies, like Extreme Programming and Scrum, embrace iterative development, continuous feedback, and adaptation. Evolutionary design, which works well with Agile, involves designing in small increments, coding, testing, and deploying frequently. This approach allows for flexibility, accommodates changing requirements, and enables faster feedback loops.

Software Key Design Methods:

□ Data Flow-Oriented Design:

Focuses on the flow of data through a system and how it's transformed at each step.

□ Data Structured Design:

Emphasizes the structure of data and how it's organized within the system.

□ Object-Oriented Design:

Organizes software around objects that encapsulate data and behavior, promoting modularity and reusability.

□ Component-Based Design:

Builds systems from reusable, independent components, allowing for easier maintenance and updates.

□ Service-Oriented Architecture (SOA):

Designs systems as a collection of loosely coupled, interoperable services.

Evolution of Design Methods:

□ Early methods: Focused on modularity, top-down design, and procedural aspects.

□ Later methods: Introduced concepts like data flow and data structure design.

□ Modern methods: Emphasize agility, continuous feedback, and adapting to changing requirements.

Importance of Design Evolution:

□ Adaptability:

Software design methods must evolve to meet the demands of increasingly complex and dynamic software

systems.

□ Efficiency:

Evolving design methods can improve development speed, reduce costs, and enhance the quality of the final product.

□ Maintainability:

Well-designed software, built using appropriate methods, is easier to maintain, update, and extend over time.

In essence, understanding the evolution of software design methods is critical for building robust, adaptable, and maintainable software solutions. Adopting agile and iterative approaches, along with a deep understanding of different design paradigms, allows developers to navigate the complexities of modern software development effectively.

SOFTWARE SYSTEM REQUIREMENTS

Software system requirements are the specifications that define what a software system should do and how it should perform. They cover both functional aspects (what the software does) and non-functional aspects (how well it does it). These requirements ensure the software meets user needs and is compatible with the intended environment.

Types of Software Requirements

Software Requirements are mainly classified into three types:

- Functional requirements
- Non-functional requirements
- Domain requirements

Functional requirements

These describe the specific actions the software must perform, such as processing orders, displaying data, or authenticating users.

Examples:

- User Authentication: The system must allow users to log in using a username and password.
- Search Functionality: The software should enable users to search for products by name or category.

□ Report Generation: The system should be able to generate sales reports for a specified date range.

Explanation: Functional requirements specify the actions that the software needs to perform. These are the basic features and functionalities that users expect from the software.

NON- FUNCTIONAL REQUIREMENTS

Non-functional requirements describe how the software performs a task rather than what it should do. They define the quality attributes, performance criteria, and constraints.

Examples:

□ Performance: The system should process 1,000 transactions per second.

□ Usability: The software should be easy to use and have a user-friendly interface.

□ Reliability: The system must have 99.9% uptime.

□ Security: Data must be encrypted during transmission and storage.

Explanation: Non-functional requirements are about the system's behavior, quality, and constraints. They ensure that the software meets certain standards of performance, usability, reliability, and security.

DOMAIN REQUIREMENTS

Domain requirements are specific to the domain or industry in which the software operates. They include terminology, rules, and standards relevant to that particular domain.

Examples:

□ Healthcare: The software must comply Health Insurance Portability and Accountability Act (HIPAA) regulations for handling patient data. HIPAA is a US federal law enacted in 1996. It establishes national standards for protecting sensitive patient health information from disclosure without the patient's consent or knowledge. HIPAA also aims to improve the efficiency and effectiveness of the healthcare system by standardizing certain electronic transactions.

□ Finance: The system should adhere Generally Accepted Accounting Principles (GAAP). GAAP is a set of accounting rules, procedures, and standards that companies in the United States must follow when preparing their financial statements. The purpose of GAAP is to ensure consistency, comparability, and transparency in financial reporting, making it easier for investors and other stakeholders to understand and analyze a company's financial performance.

□ E-commerce: The software should support various payment gateways like PayPal, Stripe, and credit cards.

Explanation: Domain requirements reflect the unique needs and constraints of a particular industry. They ensure that the software is relevant and compliant with industry-specific regulations and standards.

OTHER ASPECTS OF SOFTWARE REQUIREMENTS INCLUDE:

□ Hardware Requirements:

These specify the necessary hardware components, like CPU speed, memory (RAM), storage space, and graphics capabilities, that a computer must have to run the software.

□ Software Requirements:

These define the compatible operating systems, drivers, and other software dependencies.

□ Interface Requirements:

These detail how the software interacts with other systems, including hardware interfaces, network protocols, and external APIs.

□ Design Constraints:

These are limitations or restrictions on the design and development of the software, such as performance targets or memory usage limits.

IMPORTANCE OF REQUIREMENTS:

□ Clarity and Communication:

Requirements documents, like the Software Requirements Specification (SRS), provide a clear understanding of the software's intended functionality and performance for developers, stakeholders, and users.

□ Development Guidance:

They act as a blueprint for developers, guiding them throughout the development process.

□ Testing and Validation:

Requirements help testers create test cases to ensure the software meets the specified criteria.

□ Risk Mitigation:

By clearly defining expectations, requirements help reduce the risk of misunderstandings and project failures.

In essence, software system requirements are the foundation for building a successful software product.

FEASIBILITY STUDY: DEFINITIONS AND METHODS.

A feasibility study is a comprehensive analysis and evaluation of a proposed project or venture to determine its practicality, viability, and potential for success. It assesses various factors to help stakeholders decide whether to proceed, modify, or abandon the project.

Key aspects of a feasibility study:

□ Assessment of practicality:

Determines if the project is achievable based on available resources, technology, and expertise.

□ Viability analysis:

Examines the project's potential for success, considering factors like market demand, financial returns, and operational requirements.

□ Risk identification:

Identifies potential challenges, limitations, and risks that could hinder the project's progress.

□ Decision-making tool:

Provides essential information for stakeholders to make informed decisions about whether to invest time, money, and resources.

Common types of feasibility studies:

□ Technical feasibility: Assesses the availability of required technology and resources.

□ Financial feasibility: Examines the project's profitability, funding sources, and cost-effectiveness.

□ Market feasibility: Evaluates the potential market for the product or service.

□ Operational feasibility: Assesses the organization's capacity to manage and execute the project.

□ Legal feasibility: Examines the project's compliance with relevant laws and regulations.

Benefits of conducting a feasibility study:

□ Informed decision-making:

Helps stakeholders make well-informed decisions based on a thorough analysis of the project's potential.

□ Risk mitigation:

Identifies potential risks early on, allowing for proactive mitigation strategies.

□ Resource optimization:

Helps optimize the use of resources by focusing on viable projects.

□ Cost savings:

Prevents costly mistakes by identifying potential problems before they escalate.

□ Improved project planning:

Provides a solid foundation for developing a comprehensive business plan and project implementation strategy.

In essence, a feasibility study is a crucial step in the project management process, helping to ensure that resources are allocated wisely and that the project has a high probability of success.

breakdown of common methods used in feasibility studies:

1. Preliminary Analysis:

□ Define Project Scope and Objectives:

Clearly outlining the project's goals, deliverables, and timeline is crucial for focus and relevance.

□ Gather Information:

This involves collecting data about the project's environment, including market conditions, potential customers, and existing resources.

☐ Identify Potential Issues:

Brainstorming potential problems, challenges, and risks associated with the project is essential for proactive mitigation.

2. Market Feasibility:

☐ Market Research:

This involves analyzing market demand, competition, and trends to determine if there's a viable market for the project.

☐ Target Audience Analysis:

Understanding the needs, preferences, and characteristics of the target audience is critical for tailoring the project effectively.

☐ Competitive Analysis:

Assessing the strengths and weaknesses of competitors helps identify opportunities and potential challenges.

3. Technical Feasibility:

☐ Resource Assessment:

Evaluating the availability of necessary technology, equipment, and skilled personnel to execute the project.

☐ Technology Evaluation:

Assessing the suitability and compatibility of chosen technologies with the project requirements.

☐ Infrastructure Review:

Determining the availability and adequacy of necessary infrastructure like power, internet, and transportation.

4. Economic Feasibility:

☐ Cost Estimation:

Accurately estimating all project costs, including development, operation, and maintenance.

☐ Revenue Projections:

Forecasting potential revenue streams and profitability based on market analysis and sales projections.

☐ Financial Analysis:

Analyzing financial viability through metrics like return on investment (ROI), net present value (NPV), and payback period.

5. Operational Feasibility:

□ Process Analysis:

Evaluating the project's operational processes and workflow to ensure efficiency and effectiveness.

□ Resource Allocation:

Determining the optimal allocation of resources, including personnel, equipment, and materials.

□ Risk Assessment and Mitigation:

Identifying potential operational risks and developing strategies to minimize their impact.

6. Legal and Regulatory Feasibility:

□ Compliance Review: Ensuring the project complies with all relevant laws, regulations, and licensing requirements.

□ Risk Assessment: Identifying and assessing potential legal and regulatory risks.

□ Permitting and Approvals: Determining the necessary permits and approvals required to operate the project.

7. Reporting and Decision-Making:

□ Documentation: Organizing and presenting findings in a clear and concise report.

□ Recommendations: Providing well-supported recommendations based on the feasibility study results.

□ Decision Making: Using the findings to make informed decisions about whether to proceed with the project, modify it, or abandon it.

DESIGNING SOFTWARE INTERFACE

Software interface design focuses on creating user-friendly and visually appealing interfaces for software applications. This involves designing the way users interact with the software, ensuring it's intuitive, efficient, and enjoyable.

Key aspects include:

layout

navigation

visual elements and user feedback.

Core Principles of Software Interface Design:

- User-centered design: Prioritizing the user's needs and goals throughout the design process.
- Consistency: Maintaining a consistent look and feel across the application to aid user navigation.
- Clarity and simplicity: Ensuring the interface is easy to understand and use, avoiding unnecessary complexity.
- Feedback: Providing clear and immediate feedback to user actions, such as button clicks or data input.
- Efficiency: Designing for quick and easy task completion, reducing user effort.
- Accessibility: Making the interface usable by people with disabilities, including those with visual or motor impairments.

Key Steps in the Design Process:

1. Gathering Requirements: Understanding the purpose of the software and the needs of its users.
2. User Analysis: Studying user behaviors, needs, and preferences to inform the design.
3. Wire framing: Creating basic outlines of the interface to plan the layout and navigation.
4. Prototyping: Developing interactive models of the interface to test functionality and usability.
5. Testing: Evaluating the interface with real users to identify areas for improvement.
6. Refinement: Making adjustments to the design based on testing results.

Essential Design Elements:

- Typography: Choosing readable and appropriate fonts for text elements.
- Color Schemes: Selecting colors that are visually appealing and support readability.
- Navigation: Designing intuitive ways for users to move between different parts of the application.
- Buttons and Forms: Creating clear and functional buttons and forms for user input.
- Layout: Organizing content and elements in a logical

and visually appealing manner.

Tools for Interface Design:

Figma:

A popular collaborative interface design tool for creating prototypes and mockups.

Sketch:

A vector-based design tool, widely used for UI design, particularly for Mac users.

Adobe XD:

A vector-based UI/UX design tool by Adobe, enabling prototyping and collaboration.

Axure RP:

A tool for creating interactive prototypes and documentation for desktop applications.

Balsamiq:

A wireframing tool focused on low-fidelity prototypes to facilitate early design

UXPin:

A platform for UI design and prototyping with a focus on design system consistency.

Marvel:

A web-based tool for creating prototypes from static designs, often used for quick prototyping.

Adobe Illustrator:

A versatile tool for creating visual elements and illustrations for interfaces.